

```

// Vulkan 1.4 INT16 및 INT8 스토리지 피처 활성화 설정
VkPhysicalDeviceShaderFloat16Int8Features int8Features{};
int8Features.sType =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT16_INT8_FEATURES;
int8Features.shaderInt8 = VK_TRUE;

VkPhysicalDevice16BitStorageFeatures int16StorageFeatures{};
int16StorageFeatures.sType =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_16BIT_STORAGE_FEATURES;
int16StorageFeatures.storageBuffer16BitAccess = VK_TRUE;

VkPhysicalDeviceFeatures2 deviceFeatures2{};
deviceFeatures2.sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2;
deviceFeatures2.pNext = &int8Features;
int8Features.pNext = &int16StorageFeatures;

#version 460
#extension GL_EXT_shader_16bit_storage : require
#extension GL_EXT_shader_explicit_arithmetic_types_int8 : require
#extension GL_EXT_shader_explicit_arithmetic_types_int16 : require

layout(local_size_x = 256, local_size_y = 1, local_size_z = 1) in;

// INT16 스토리지 버퍼 (16비트 공간 안에 INT8 두 개가 중첩 패킹됨)
layout(set = 0, binding = 0) readonly buffer InputBuffer16 {
    int16_t packedInt16Data[];
};

layout(set = 0, binding = 1) writeonly buffer OutputBuffer32 {
    int32_t resultData[]; // 최종 연산 결과 (오버플로 방지를 위해 32비트 누산)
};

void main() {
    uint idx = gl_GlobalInvocationID.x;

    // 1. 16비트 데이터 로드 및 두 개의 INT8 값으로 분리 (비트 시프트 및 마스킹)
    int16_t raw16 = packedInt16Data[idx];
    int8_t val_low = int8_t(raw16 & 0x00FF);
    int8_t val_high = int8_t((raw16 >> 8) & 0x00FF);

    // 2. INT32로 승격 후 RDNA3 INT16/INT8 SIMD 유닛 레이어에서 연산 수행
    // 예: 두 패킹된 정수 페어의 곱셈 및 누산 (Dot Product 성격의 연산)
    int32_t a = int32_t(val_low);
    int32_t b = int32_t(val_high);
    int32_t computed = (a * b) + (a + b);

    // 3. 최종 결과 스토어
    resultData[idx] = computed;
}

```

